

Alonline

# MySQL BR CONF 2026

## About Me

Database Administrator in **DBAOnline**

(MySQL, MongoDB, PostgreSQL e SQL Server)

- Oracle ACE Associate (MySQL)
- MySQL Database 8.0 Administrator
- MySQL Database 8.0 Developer
- MySQL Heatwave Implementation
- OCI Foundations 2025
- MySQL Solution Engineering Specialist



**Henrique Garcia**

# What are we going to talk about today...

**MVCC (Multi-Version Concurrency Control ) is not magic!!**

```
START TRANSACTION;  
  
DROP TABLE sbtest3;  
  
ROLLBACK;
```

In the situation above, will the **ROLLBACK** restore my table?

So... We validated:

```
mysql> SELECT database();
+-----+
| database() |
+-----+
| comunidademysql |
+-----+
1 row in set (0.03 sec)

mysql> SHOW TABLES;
+-----+
| Tables_in_comunidademysql |
+-----+
| sbtest1 |
| sbtest2 |
| sbtest3 |
+-----+
3 rows in set (0.03 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.03 sec)

mysql> DROP TABLE sbtest3;
Query OK, 0 rows affected (0.11 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.03 sec)

mysql> SHOW TABLES;
+-----+
| Tables_in_comunidademysql |
+-----+
| sbtest1 |
| sbtest2 |
+-----+
2 rows in set (0.03 sec)

mysql> |
```

**DBA**<sup>®</sup>  
ONLINE



 MongoDB.

# What do we expect from a transaction?

When it comes to transactions, what comes to mind?

- **INSERT**
- **UPDATE**
- **DELETE**
- **COMMIT**
- **ROLLBACK**

## What does this teach us?

“Everything that happens inside a transaction can be undone.”

```
mysql> SELECT id, k FROM sbtest2 WHERE id = 1;
+-----+
| id | k      |
+-----+
| 1  | 504993 |
+-----+
1 row in set (0.04 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (5.96 sec)

mysql> UPDATE sbtest2 SET k = 123456 WHERE id = 1;
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT NOW(); SELECT id, k FROM sbtest2 WHERE id = 1;
+-----+
| NOW() |
+-----+
| 2026-09-12 11:13:55 |
+-----+
1 row in set (0.03 sec)

+-----+
| id | k      |
+-----+
| 1  | 123456 |
+-----+
1 row in set (0.03 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.03 sec)

mysql> SELECT NOW(); SELECT id, k FROM sbtest2 WHERE id = 1;
+-----+
| NOW() |
+-----+
| 2026-09-12 11:14:49 |
+-----+
1 row in set (0.03 sec)

+-----+
| id | k      |
+-----+
| 1  | 504993 |
+-----+
1 row in set (0.03 sec)

mysql> |
```

# Why do we believe this?

## ***ACID.***

- **A → *Atomicity***
  - All operations succeed or none of them do.
- **C → *Consistency*.**
  - The database remains in a valid state.
- **I → *Isolation*.**
  - Transactions do not interfere improperly with each other.
- **D → *Durability*.**
  - Committed changes persist.

**Atomicity** leads us to believe that everything inside a **transaction** can be **rolled back**.

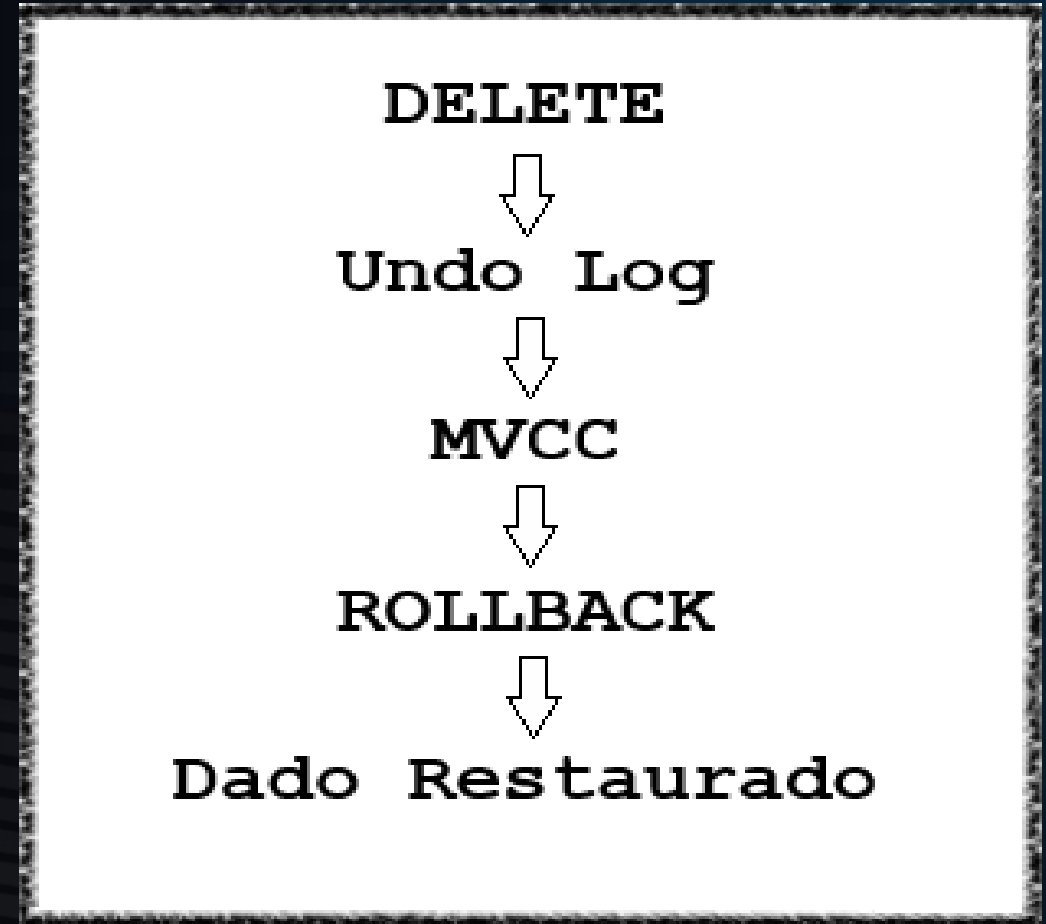
# Where does MVCC actually work?

**MVCC** works with:

- **INSERT**
- **DELETE**
- **UPDATE**
- **SELECT**

Therefore:

- ✓ **MVCC** was designed to protect **DATA**.





# But does the same logic apply to DDL?

When we execute:

```
mysql>  
mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO sbtest2 (k, c, pad) VALUES (23, 'Henrique', 'Garcia');  
Query OK, 1 row affected (0.04 sec)  
  
mysql> ALTER TABLE sbtest2 ADD COLUMN uf CHAR(2);  
Query OK, 0 rows affected (0.40 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql> ROLLBACK;  
Query OK, 0 rows affected (0.04 sec)  
  
mysql> |
```

When we validate:

```
mysql> SELECT * FROM sbtest2 WHERE id = 1000001;  
+-----+-----+-----+-----+-----+  
| id      | k  | c      | pad   | uf    |  
+-----+-----+-----+-----+-----+  
| 1000001 | 23 | Henrique | Garcia | NULL  |  
+-----+-----+-----+-----+-----+  
1 row in set (0.05 sec)
```



# The Catch: DDL does not follow the same rules.

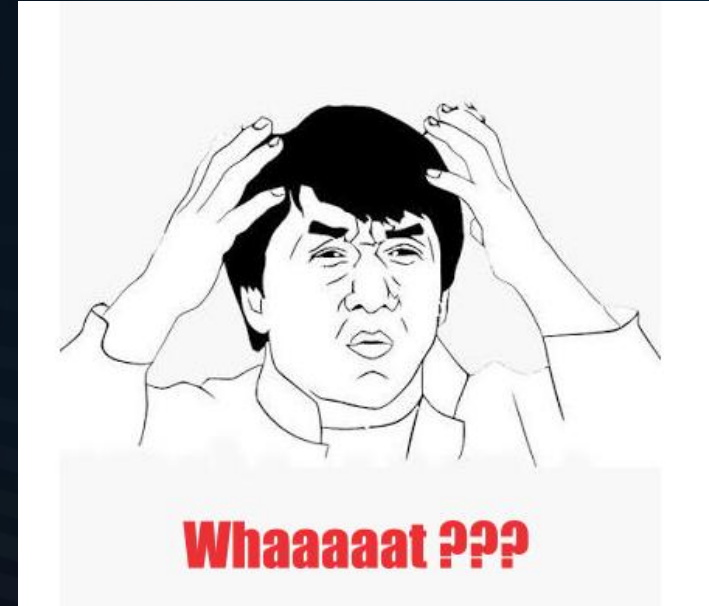
Following the same line of reasoning, we know that:

```
START TRANSACTION;  
  
UPDATE sbtest2 SET k = 123456 WHERE id = 1;  
  
ROLLBACK;
```

✓ The data will be restored.

And why not expect the same from this:

```
START TRANSACTION;  
  
INSERT INTO;  
  
ALTER TABLE sbtest2 ADD COLUMN;  
  
ROLLBACK;
```

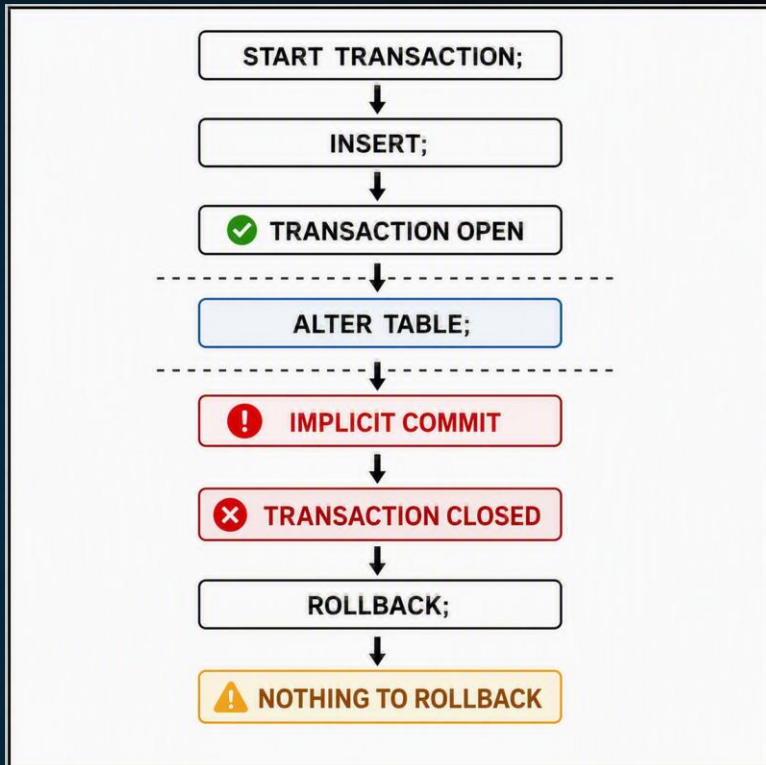


# The real culprit: Implicit Commit...

When we **executed**:

```
START TRANSACTION;  
INSERT INTO;  
ALTER TABLE sbtest2 ADD COLUMN;  
ROLLBACK;
```

MySQL will do something similar to:



**Result:**

- The **active transaction** is automatically **ended**.
- The **DDL command runs** in its **own transaction**.
- The **Rollback** has **nothing left to undo**.

**Main commands:**

- ALTER TABLE.
- DROP TABLE.
- CREATE TABLE.
- RENAME TABLE.
- TRUNCATE TABLE.

# Will Atomic DDL solve my problem?

When Atomic DDL arrived in MySQL 8, a question arose:

*"If MySQL 8 has Atomic DDL, does that mean ALTER, DROP and TRUNCATE now participate in a transaction?"*

```
ALTER TABLE sbtest2  
ADD COLUMN email VARCHAR(50);
```

If the server failed during a DDL operation, the database could be left in an inconsistent state.

Atomic DDL works as follows:

- ✓ Operation Completed.
- ✓ Operation Internally Rolled Back.

**ATOMIC DDL** means:

- ✓ **Protection against server failures.**
- ✓ **More consistent operations.**
- ✓ **Consistent metadata.**

**ATOMIC DDL** does **NOT** mean:

- ❖ **Manual rollback.**
- ❖ **Participation in MVCC.**
- ❖ **Controlled by START TRANSACTION.**

# Key Takeaways from This Session?

**MVCC** protects **DATA**:

- ✓ **INSERT**
- ✓ **UPDATE**
- ✓ **DELETE**
- ✓ **SELECT**

MVCC uses the **Undo Log** to maintain data versions:

- ✓ **Consistent reads.**
- ✓ **Rollback.**
- ✓ **Concurrency.**

**MVCC protects DATA, not DATABASE structures.**

**DDL** does not participate in MVCC:

- ❖ **ALTER TABLE**
- ❖ **DROP TABLE**
- ❖ **TRUNCATE TABLE**
- ❖ **CREATE TABLE**

Causing an **IMPLICIT COMMIT**.

- ✓ **Atomic DDL** improved reliability.
- ✓ **Atomic DDL** does **NOT** mean rollback.

# Blog MySQL – Last Posts!



1º - I deleted rows but the table size didn't change? See why.



2º - How MVCC and the Undo Log work in MySQL InnoDB.



3º - What is the MySQL InnoDB Purge Thread and why does it stall?



4º - How to reclaim disk space with OPTIMIZE TABLE in MySQL.

# Questions???

LinkedIn



Oracle ACE



Thank you all!!!

**FLYNNNA**®  
BY ALLONLINE

*Allonline*

**DBA**®  
ONLINE